

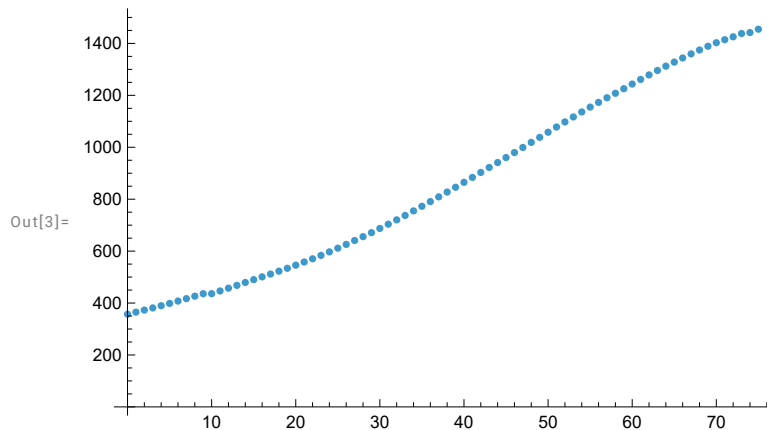
```
In[1]:= india = TimeSeries@Transpose[N@{{1, 0}, {0, 10-6}}.  
      Transpose[Import[NotebookDirectory[] <> "../data/ipop.csv", "CSV"]] - {1950, 0}]
```

```
Out[1]= TimeSeries[ Time: 0. to 75.  
Data points: 76]
```

```
In[2]:= p0 = india[0]
```

```
Out[2]= 357.021
```

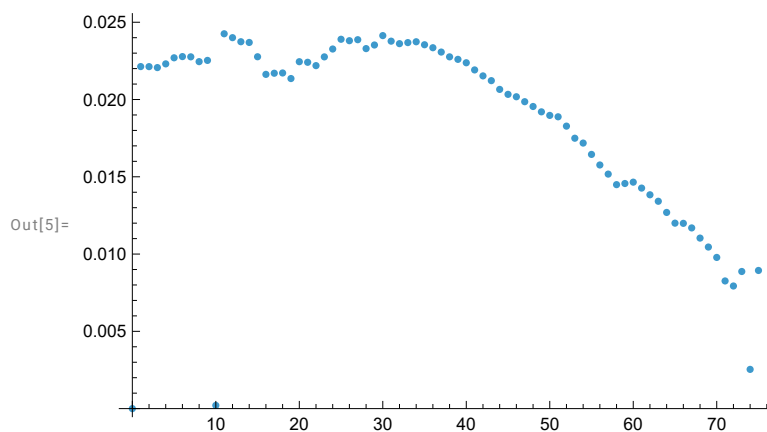
```
In[3]:= ListPlot[india]
```



```
In[4]:= gr = TimeSeries[  
      Transpose[{  
        india["Times"],  
        {0} ~ Join~ (Differences[india["Values"]] / india["Values"]) [[;; -2]]  
      }]]
```

```
Out[4]= TimeSeries[ Time: 0. to 75.  
Data points: 76]
```

```
In[5]:= ListPlot[gr]
```



Trying r = average

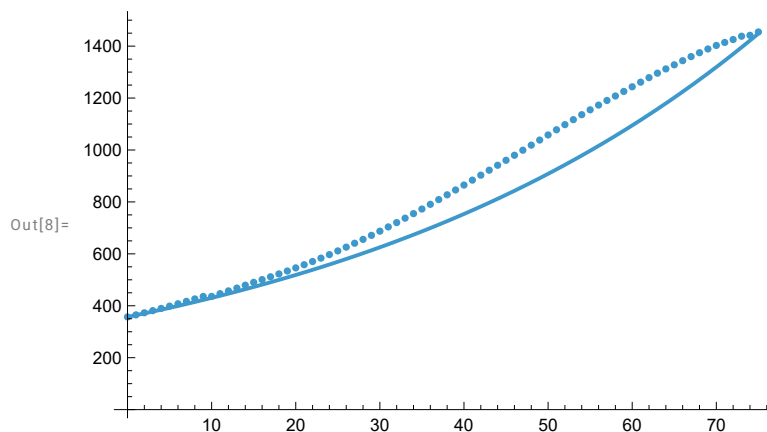
```
In[6]:= r[1] = gr["Values"] // Mean // N
```

```
Out[6]= 0.018672
```

```
In[7]:= s[1] = DSolve[{p'[t] == r[1] × p[t], p[0] == p0}, p[t], t]
```

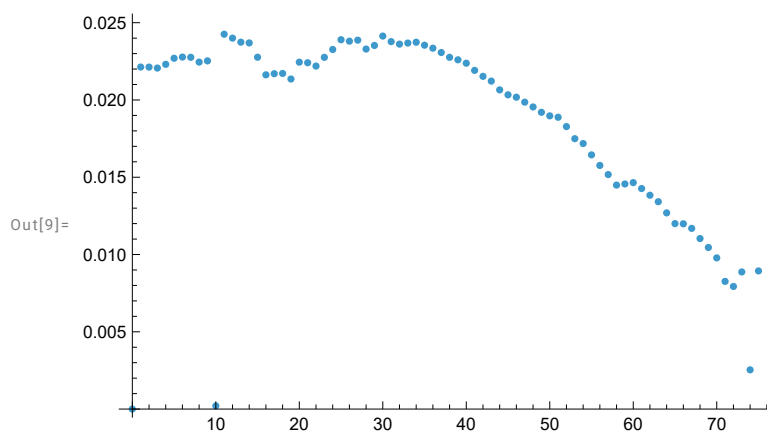
```
Out[7]= {{p[t] → 357.021 e0.018672 t}}
```

```
In[8]:= Show[
  ListPlot[india],
  Plot[p[t] /. s[1], {t, 0, 75}]
]
```



Let's try to fit a linear curve for $r(t)$. Recall:

```
In[9]:= ListPlot[gr]
```

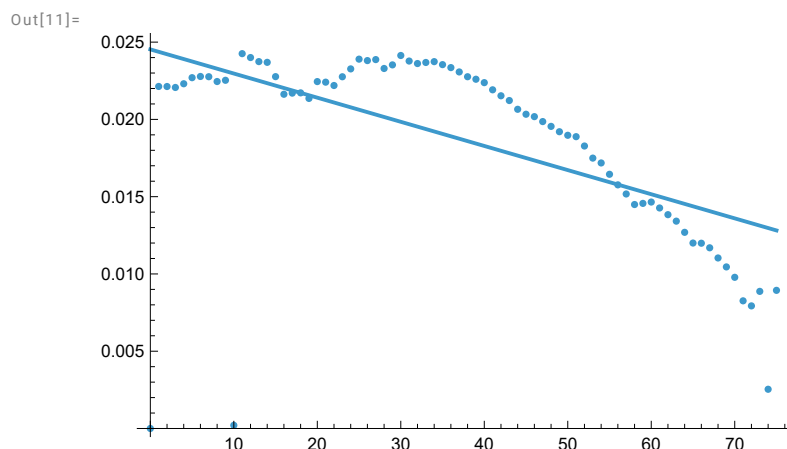


```
In[10]:= r[2] = NonlinearModelFit[gr["Path"], a0 + a1 t, {a0, a1}, t]
```

```
Out[10]=
```

```
FittedModel[ 0.0245 - 0.000156 t ]
```

```
In[11]:= Show[
  ListPlot[gr],
  Plot[r[2][t], {t, 0, 75}]]
```

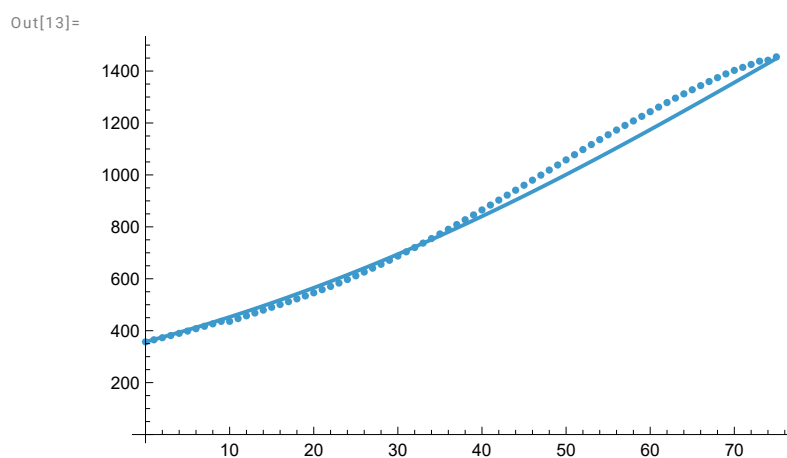


```
In[12]:= s[2] = DSolve[{p'[t] == r[2][t] * p[t], p[0] == p0}, p[t], t]
```

Out[12]=

$$\left\{ \left\{ p[t] \rightarrow 357.021 e^{0.0245294 t - 0.0000780979 t^2} \right\} \right\}$$

```
In[13]:= Show[
  ListPlot[india],
  Plot[p[t] /. s[2], {t, 0, 75}]
]
```



which is a far better fit! notice that **gr** looks more like a quadratic than a linear, so we can do

```
In[14]:= r[3] = NonlinearModelFit[gr["Path"], a0 + a1 t + a2 t^2, {a0, a1, a2}, t]
```

Out[14]=

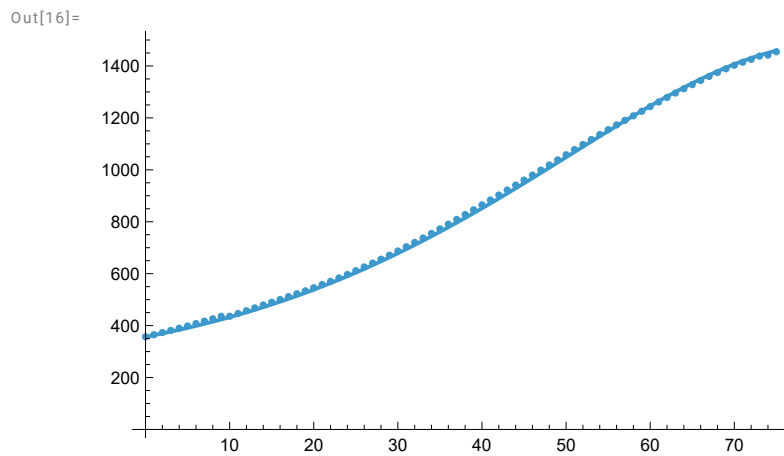
$$\text{FittedModel} \left[0.0172 + 0.000435 t - 7.88 \times 10^{-6} t^2 \right]$$

```
In[15]:= s[3] = DSolve[{p'[t] == r[3][t] * p[t], p[0] == p0}, p[t], t]
```

Out[15]=

$$\left\{ \left\{ p[t] \rightarrow 357.021 e^{0.0172383 t + 0.000217484 t^2 - 2.62739 \times 10^{-6} t^3} \right\} \right\}$$

```
In[16]:= Show[
  ListPlot[india],
  Plot[p[t] /. s[3], {t, 0, 75}]
]
```



amazing.

next, we try logistics growth model with the above 3 growth rate approximations.

```
In[55]:= pops = india["Values"];
```

```
In[56]:= mod[1][k_?NumberQ] := (mod[1][k] =
  First[p /. NDSolve[{p'[t] == r[1] * p[t] (1 - p[t] / k), p[0] == p0}, p, {t, 0, 100}]]);
```

```
In[57]:= fit = FindFit[pops, mod[1][k][p], {{k, 2500}}, p, PrecisionGoal -> 4, AccuracyGoal -> 4]
```

Out[57]=

$$\{k \rightarrow 2.95499 \times 10^8\}$$

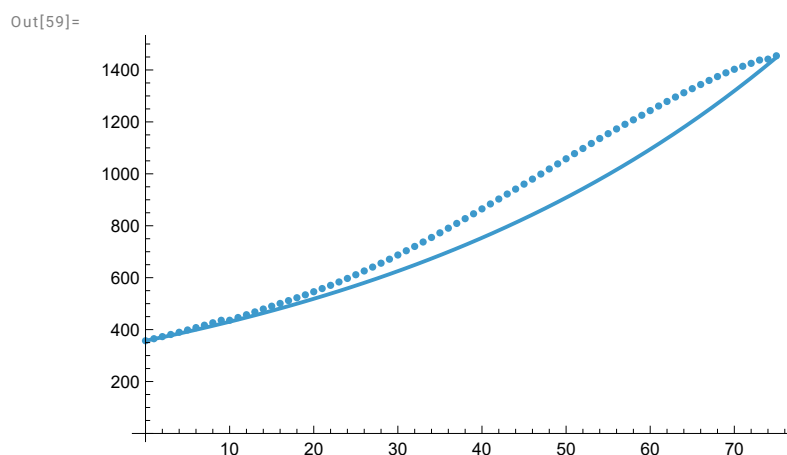
solving with this value we get:

```
In[58]:= s[4] = NDSolve[
  {p'[t] == r[1] * p[t] (1 - p[t] / 2.954989520043017`*^8), p[0] == p0}, p, {t, 0, 100}]
```

Out[58]=

$$\left\{ \left\{ p \rightarrow \text{InterpolatingFunction} \left[\left\{ \begin{array}{c} \text{Domain: } \{0., 100.\} \\ \text{Output: scalar} \end{array} \right\} \right] \right\} \right\}$$

```
In[59]:= Show[
  ListPlot[india],
  Plot[p[t] /. s[4], {t, 0, 75}]
]
```



not great! let's try linear and quadratic rates.

```
In[60]:= mod[2][k_?NumberQ] := (mod[2][k] = First[
  p /. NDSolve[{p'[t] == r[2][t] * p[t] (1 - p[t] / k), p[0] == p0}, p, {t, 0, 100}]]);
```

```
In[61]:= fit = FindFit[pops, mod[2][k][p], {{k, 2500}}, p, PrecisionGoal -> 4, AccuracyGoal -> 4]
```

Out[61]=

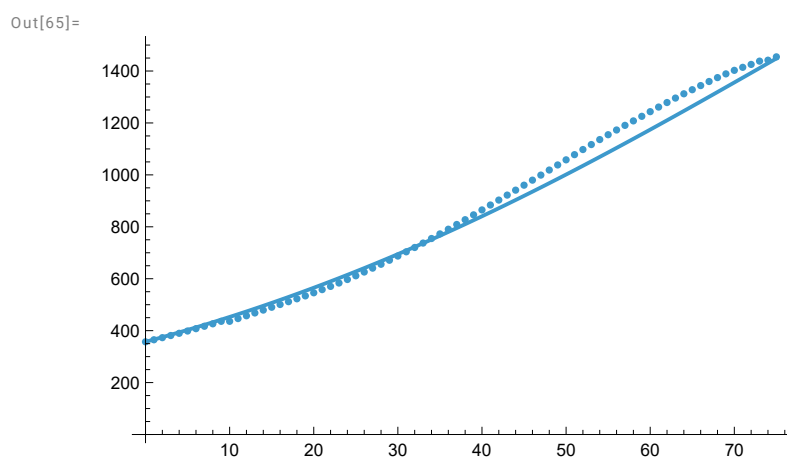
$$\{k \rightarrow 1.76324 \times 10^9\}$$

```
In[64]:= s[5] = NDSolve[
  {p'[t] == r[2][t] * p[t] (1 - p[t] / 1.763242127921628`*^9), p[0] == p0}, p, {t, 0, 100}]
```

Out[64]=

$$\left\{ \left\{ p \rightarrow \text{InterpolatingFunction} \left[\left\{ \begin{array}{c} \text{Domain: } \{0., 100.\} \\ \text{Output: scalar} \end{array} \right\} \right] \right\} \right\}$$

```
In[65]:= Show[
  ListPlot[india],
  Plot[p[t] /. s[5], {t, 0, 75}]
]
```



clearly better. finally, quadratic rate:

```
In[66]:= mod[3][k_?NumberQ] := (mod[3][k] = First[
    p /. NDSolve[{p'[t] == r[3][t] × p[t] (1 - p[t] / k), p[0] == p0}, p, {t, 0, 100}]]);
```

```
In[67]:= FindFit[pops, mod[3][k][p], {{k, 2500}}, p, PrecisionGoal → 4, AccuracyGoal → 4]
```

```
Out[67]= {k → 72501.5}
```

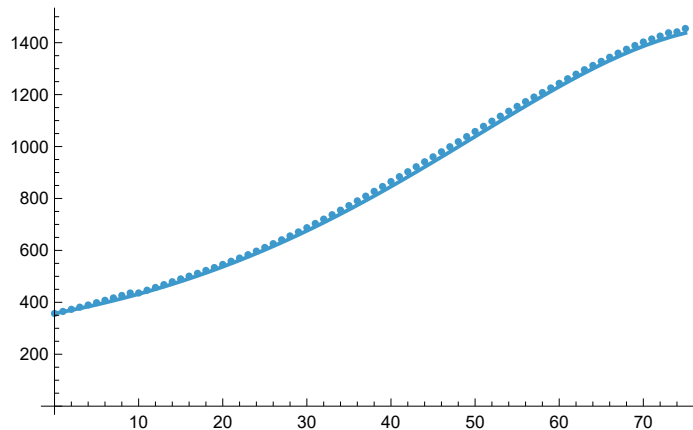
```
In[68]:= s[6] = NDSolve[
    {p'[t] == r[3][t] × p[t] (1 - p[t] / 72501.53869790983`), p[0] == p0}, p, {t, 0, 100}]
```

```
Out[68]=
```

```
{ {p → InterpolatingFunction[ Domain: {{0., 100.}} Output: scalar ] ] }
```

```
In[69]:= Show[
    ListPlot[india],
    Plot[p[t] /. s[6], {t, 0, 75}]
]
```

```
Out[69]=
```



possibly the best so far!

We now go one step further to introduce delays in the logistics growth equation:

```
In[73]:= mod[4][k_?NumberQ, τ_?NumericQ] := (mod[4][k, τ] = First[p /.
    NDSolve[{p'[t] == r[1] × p[t] (1 - p[t - τ] / k), p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]]);
```

```
In[74]:= FindFit[pops, mod[4][k, τ][p],
    {{k, 2500}, {τ, 5}}, p, PrecisionGoal → 4, AccuracyGoal → 4]
```

 **FindFit:** The step size in the search has become less than the tolerance prescribed by the PrecisionGoal option, but the gradient is larger than the tolerance specified by the AccuracyGoal option. There is a possibility that the method has stalled at a point that is not a local minimum.

```
Out[74]=
```

```
{k → 12215.4, τ → 8.13672}
```

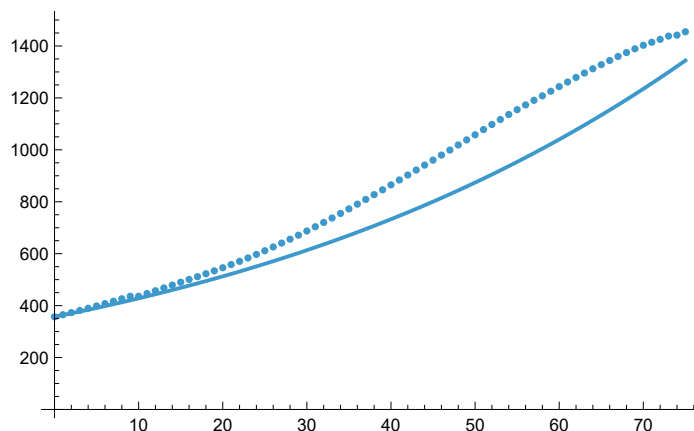
```
In[80]:= s[7] =
    NDSolveValue[{p'[t] == r[1] × p[t] (1 - p[t - 8.136716080616184`] / 12215.387039200396`),
    p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]
```

```
Out[80]=
```

```
InterpolatingFunction[ Domain: {{0., 100.}} Output: scalar ]
```

```
In[81]:= Show[
  ListPlot[india],
  Plot[s[7][t], {t, 0, 75}]
]
```

Out[81]=



clearly not better. trying other two rates:

```
In[88]:= mod[5][k_?NumberQ, τ_?NumericQ] := (mod[5][k, τ] = First[p /. NDSolve[
  {p'[t] == r[2][t] × p[t] (1 - p[t - τ] / k), p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]]);
```

In[110]:=

```
FindFit[pops, mod[5][k, τ][t],
  {{k, 2500}, {τ, 10}}, t, PrecisionGoal → 4, AccuracyGoal → 4]
```

FindFit: The step size in the search has become less than the tolerance prescribed by the PrecisionGoal option, but the gradient is larger than the tolerance specified by the AccuracyGoal option. There is a possibility that the method has stalled at a point that is not a local minimum.

Out[110]=

```
{k → 12795.4, τ → 10.1508}
```

In[113]:=

```
s[8] = NDSolveValue[
  {p'[t] == r[2][t] × p[t] (1 - p[t - 10.150812379388723] / 12795.38555507983),
  p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]
```

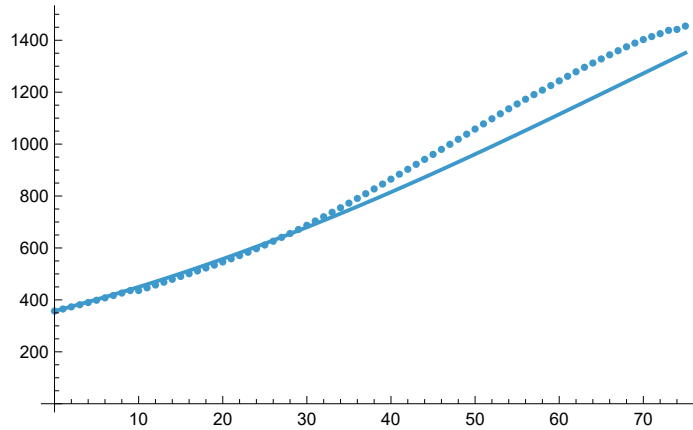
Out[113]=

```
InterpolatingFunction[ Domain: {{0., 100.}}
Output: scalar]
```

In[114]:=

```
Show[
  ListPlot[india],
  Plot[s[8][t], {t, 0, 75}]
]
```

Out[114]=



In[115]:=

```
mod[6][k_?NumberQ, τ_?NumericQ] := (mod[6][k, τ] = First[p /. NDSolve[
  {p'[t] == r[3][t] × p[t] (1 - p[t - τ] / k), p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]]);
```

In[125]:=

```
FindFit[pops, mod[6][k, τ][p],
  {{k, 2500}, {τ, 8}}, p, PrecisionGoal → 4, AccuracyGoal → 4]
```

FindFit: The step size in the search has become less than the tolerance prescribed by the PrecisionGoal option, but the gradient is larger than the tolerance specified by the AccuracyGoal option. There is a possibility that the method has stalled at a point that is not a local minimum.

Out[125]=

```
{k → 5441.1, τ → 2.98452}
```

In[126]:=

```
s[9] =
  NDSolveValue[{p'[t] == r[3][t] × p[t] (1 - p[t - 2.9845165925609534`] / 5441.09564299777`),
    p[t /; t ≤ 0] == p0}, p, {t, 0, 100}]
```

Out[126]=

```
InterpolatingFunction[  Domain: {{0., 100.}}
Output: scalar]
```


In[127]:=

```
Show[  
  ListPlot[india],  
  Plot[s[9][t], {t, 0, 75}]  
]
```

Out[127]=

